Del dato al asistente: predicción del riesgo de deserción estudiantil con Machine Learning y APIs

Chanco Capucho Fabrizzio Renzo, Quinto Condori Frederick, Quinto Morales Miguel Augusto y Salvatierra Casallo Yuan L
Estudiantes de Ing. Sistemas
Tecnologías Emergentes (ISO46B) – Grupo 1
Universidad Nacional del Centro del Perú
Huancayo, Perú

***Resumen*—El abandono universitario representa uno de los desafíos más críticos para las instituciones de educación superior en la actualidad. En este proyecto se construyó un sistema integral de clasificación multiclase diseñado para apoyar a las oficinas de tutoría en la identificación temprana de estudiantes con riesgo de deserción al finalizar su primer semestre. Se utilizó un enfoque riguroso de validación, comparando una línea base trivial con un modelo de regresión logística y tres arquitecturas de redes neuronales (Perceptrón Multicapa). Empleando ocho variables académicas y socioeconómicas cuidadosamente seleccionadas para evitar la fuga de información, la regresión logística demostró ser el modelo más estable, obteniendo un macro-F1 de 0.6514 en validación y 0.6537 en prueba. El modelo promovido se encapsuló y desplegó mediante una arquitectura backend en FastAPI, la cual expone endpoints RESTful. Finalmente, esta API es consumida por un agente conversacional (chatbot) basado en reglas, proporcionando una interfaz accesible para la recolección de datos y derivación de casos críticos. El sistema mantiene un enfoque estrictamente orientativo y ético, auditando sesgos demográficos y garantizando que las predicciones algorítmicas no reemplacen el juicio profesional de los tutores.**

***Index Terms*—deserción estudiantil, aprendizaje automático, clasificación multiclase, regresión logística, redes neuronales MLP, FastAPI, chatbot, ética de datos, ingeniería de software.**

I. Introducción

LA deserción estudiantil en el ámbito universitario no solo representa una pérdida significativa de recursos institucionales y estatales, sino que impacta profundamente en la trayectoria personal y profesional de los jóvenes. Identificar de manera temprana qué estudiantes se encuentran en una trayectoria de riesgo (*Dropout*) frente a aquellos que continuarán matriculados (*Enrolled*) o se graduarán con éxito (*Graduate*) permite a las oficinas de bienestar universitario focalizar sus recursos limitados de tutoría.

Históricamente, los sistemas de alerta temprana dependían de la reactividad: un estudiante era contactado solo después de reprobar múltiples asignaturas o acumular deudas insalvables. El advenimiento de las Tecnologías Emergentes y el Aprendizaje Automático (*Machine Learning*) ofrece la posibilidad de predecir estos eventos antes de que se vuelvan irreversibles.

El objetivo principal de este proyecto es desarrollar y desplegar un sistema predictivo de clasificación multiclase que evalúe el riesgo de abandono al cierre del primer semestre. Los objetivos específicos incluyen: 1) Realizar un análisis exploratorio y curación de datos previniendo la fuga temporal de variables; 2) Entrenar y comparar sistemáticamente modelos paramétricos y no paramétricos; 3) Desplegar el modelo ganador mediante una API robusta y documentada; 4) Desarrollar una interfaz conversacional amigable que interactúe con el modelo en tiempo real.

II. Estado del Arte

La predicción de la deserción estudiantil ha sido abordada extensamente en la literatura del análisis del aprendizaje (*Learning Analytics*). Modelos tempranos dependían fundamentalmente de regresiones lineales y análisis de supervivencia. Sin embargo, la complejidad multivariante del rendimiento académico ha propiciado la transición hacia modelos de ensamble y redes neuronales profundas.

Estudios recientes utilizando el *Predict Students' Dropout and Academic Success Dataset* han demostrado que variables de índole económica (como estar al día en la matrícula o ser becario) tienen un impacto predictivo tan fuerte como el rendimiento académico puro. No obstante, muchos de estos trabajos cometen el error metodológico de incluir el historial académico completo de la carrera para predecir si el estudiante se graduará, lo cual carece de utilidad práctica preventiva (fuga temporal). En contraste, la presente investigación restringe la ventana de observación estrictamente al primer semestre, priorizando la utilidad operativa sobre la exactitud teórica infundada.

III. Consideraciones Éticas y Privacidad

La integración de inteligencia artificial en la toma de decisiones educativas conlleva riesgos sustanciales si no se establecen guardarraíles éticos claros.

III-A. El rol del algoritmo

El sistema diseñado actúa como un mecanismo de priorización, no de condena. La predicción probabilística de la clase *Dropout* no debe ser utilizada en ninguna circunstancia para justificar sanciones disciplinarias, condicionar el acceso a financiamiento o etiquetar peyorativamente al estudiante.

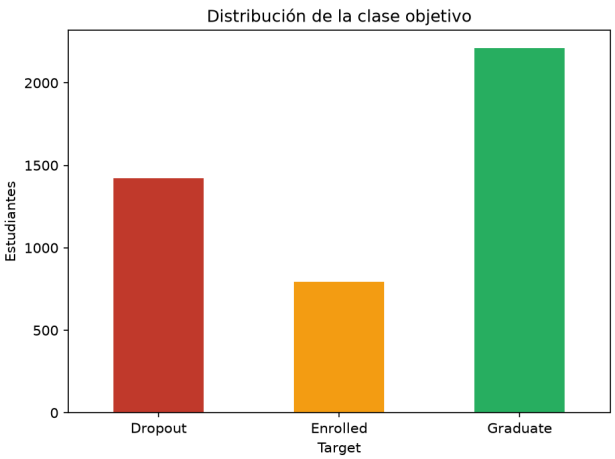


Figura 1. Distribución desbalanceada de la clase objetivo.

III-B. Equidad y variables protegidas

A nivel de modelado, se excluyeron explícitamente atributos demográficos sensibles como el género, el estado civil, la nacionalidad y las necesidades educativas especiales de las variables de entrada (*features*). Incluirlos podría generar que el modelo aprenda y perpetúe sesgos históricos. Sin embargo, la simple exclusión no garantiza la equidad. Por ello, estas variables se reservaron para realizar auditorías *post-hoc* del rendimiento del modelo entre distintos subgrupos.

III-C. Manejo de la privacidad

A nivel de despliegue, la arquitectura del chatbot y de la API no registra identificadores personales (DNI, nombres, correos). La inferencia se realiza al vuelo y el *payload* JSON se destruye al finalizar la sesión HTTP. Además, ante predicciones de alto riesgo o errores de interacción reiterados, el chatbot aplica una política de "falla segura", recomendando el contacto directo con la oficina de tutoría en lugar de emitir diagnósticos automatizados severos.

IV. Análisis Exploratorio y Curación de Datos

El proyecto utiliza el conjunto de datos de dominio público de la Universidad de Portalegre (UCI ID 697), compuesto originalmente por 4,424 instancias y 36 características distribuidas entre datos demográficos, socioeconómicos, macroeconómicos y académicos.

IV-A. Distribución del Objetivo

El problema se plantea como una clasificación multiclase. La distribución inicial (Figura 1) muestra:

- *Graduate* (Graduado): 2,209 registros (49.93 %)
- *Dropout* (Deserción): 1,421 registros (32.12 %)
- *Enrolled* (En curso): 794 registros (17.95 %)

El clasificador trivial que predice siempre *Graduate* obtendría casi un 50 % de *accuracy*. Por este motivo, la exactitud global se descarta como métrica principal, adoptándose el **macro-F1** y el **Recall** de la clase minoritaria y crítica (*Dropout*).

IV-B. Prevención de Fuga Temporal (Data Leakage)

Durante la exploración, se identificó que las variables del segundo semestre presentaban una separación casi perfecta de las clases. No obstante, utilizar datos del segundo semestre para predecir si un estudiante requiere ayuda *al final del primer semestre* constituye una violación causal. Consecuentemente, las seis columnas referentes al segundo semestre fueron eliminadas del *pipeline*.

IV-C. Selección del Espacio de Características

Tras la limpieza sistemática (cero valores nulos y sin duplicados conflictivos a nivel macro), se establecieron ocho características predictoras:

1. Unidades curriculares inscritas (1er Sem.)
2. Unidades curriculares aprobadas (1er Sem.)
3. Promedio ponderado (1er Sem.)
4. Estado de pago de matrícula (1/0)
5. Condición de deudor institucional (1/0)
6. Beneficiario de beca (1/0)
7. Edad al momento de ingreso (Numérico)
8. Nota o puntaje de admisión (Numérico)

Variables como la condición de deudor y el promedio del primer semestre exhiben variaciones notables entre las poblaciones que desertan y las que se gradúan.

V. Marco Metodológico de Modelado

El *pipeline* de Machine Learning se programó en Python empleando la librería `scikit-learn`. Para garantizar una evaluación insesgada, el conjunto de datos fue sometido a una partición estratificada (70 % Entrenamiento, 15 % Validación, 15 % Prueba), asegurando que la proporción original de las clases se mantuviera en cada subconjunto mediante una semilla generadora seudoaleatoria fija (`random_state=42`).

V-A. Regresión Logística Multiclase

El modelo de Regresión Logística generalizado opera bajo un enfoque *One-vs-Rest* (OvR) o multinomial. Utilizando la función *Softmax*, la probabilidad de que una observación x_i pertenezca a la clase k se define matemáticamente como:

$$P(Y = k \mid X = x_i) = \frac{e^{\beta_k \cdot x_i}}{\sum_{j=1}^{K} e^{\beta_j \cdot x_i}} \tag{1}$$

Para contrarrestar el desbalance de clases observado en el EDA, se implementó la penalización inversa a las frecuencias de clase (`class_weight='balanced'`). De este modo, los errores cometidos al clasificar la minoría (*Enrolled*) penalizan más severamente la función de costo *log-loss* (Entropía Cruzada):

$$\mathcal{L} = -\sum_{i=1}^{N}\sum_{k=1}^{K} w_k \, y_{i,k} \log(\hat{p}_{i,k}) \tag{2}$$

Tabla I
RESULTADOS COMPARATIVOS EN LA PARTICIÓN DE VALIDACIÓN

Modelo	Macro-F1 (Val)	Recall Dropout
Dummy (Baseline)	0.2222	0.0000
Regresión Logística	**0.6514**	0.6432
MLP Base	0.6348	**0.7324**
MLP Regularizada	0.6134	0.7465
MLP Profunda	0.6069	0.7371

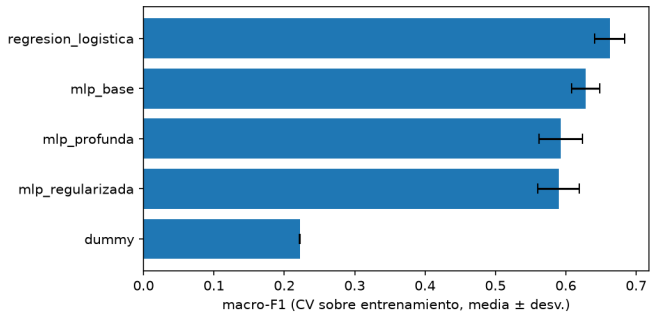


Figura 2. Estabilidad del macro-F1 validado por iteraciones repetidas sobre el subconjunto de entrenamiento.

V-B. *Perceptrón Multicapa (MLP)*

Se diseñaron tres variaciones arquitectónicas de Redes Neuronales Artificiales tipo *Feed-Forward*:

- **MLP Base:** Arquitectura somera de pocas capas ocultas.
- **MLP Regularizada:** Incorporación de penalización L2 profunda (*weight decay*) con $\alpha = 0.01$ y la técnica de parada temprana (*early stopping*) validando contra el 10 % de la partición de entrenamiento para prevenir el sobreajuste masivo.
- **MLP Profunda:** Multiplicación de la cantidad de neuronas y capas ocultas, utilizando activación ReLU: $f(x) = \max(0, x)$.

El escalado es obligatorio para ambos modelos (basados en descenso de gradiente). Se aplicó `StandardScaler`: $z = (x-\mu)/\sigma$. Crucialmente, los tensores de μ y σ se ajustaron (*fit*) exclusivamente en entrenamiento, y se aplicaron (*transform*) mecánicamente a validación y prueba.

VI. RESULTADOS Y SELECCIÓN DE MODELOS

La fase de selección (*Model Selection*) se rige estrictamente por los resultados en la partición de Validación. La métrica de decisión principal fue el Macro-F1 Score, complementada con la observación minuciosa del Recall de *Dropout*.

VI-A. *Análisis de Variabilidad en Entrenamiento*

Para verificar empíricamente la estabilidad de los algoritmos frente a distintas configuraciones iniciales, se midió la variabilidad mediante validación cruzada ($k = 5$ con 5 repeticiones). La Figura 2 ilustra que la Regresión Logística no solo obtiene medias más altas en Macro-F1 (0.6623 ± 0.0212), sino que la barra de error es comparativamente estrecha frente a la inestabilidad estocástica de los MLP.

VI-B. *Justificación de la Selección (Trade-offs)*

Como se observa en la Tabla I, los modelos neuronales (MLP) capturaron un porcentaje mayor de estudiantes en deserción (Recall Dropout > 0.73). No obstante, este incremento se logró a expensas de un colapso en el reconocimiento de la clase intermedia *Enrolled* (cuyo recall cayó a niveles en torno a 0.25).

La Regresión Logística, por el contrario, mantuvo un desempeño balanceado a lo largo de las tres categorías. Dado que promover falsos positivos excesivos sobrecargaría los recursos limitados de la oficina de tutoría, la Regresión Logística fue formalmente promovida como la versión final y serializada a formato binario (`.joblib`).

VI-C. *Evaluación Final en Prueba (Hold-out Test)*

Una vez cerrado el modelo, se expuso a la partición independiente de prueba (Figura 3), obteniendo:

- **Macro-F1:** 0.6537
- **Recall Dropout:** 0.5915
- **Recall Enrolled:** 0.6723
- **Recall Graduate:** 0.7500

VI-D. *Auditoría de Equidad (Fairness)*

Evaluando el modelo promovido frente a subgrupos protegidos en la partición de prueba, se detectaron disparidades. El modelo tiene mayor facilidad para detectar el riesgo de deserción en estudiantes no becados (0.6091) frente a becados (0.3750). Esto corrobora el riesgo ético documentado en la Sección III y hace indispensable la intervención de tutores humanos que contextualicen el puntaje de riesgo del algoritmo.

VII. INGENIERÍA DE SOFTWARE: DESARROLLO DE LA API RESTFUL

Un modelo en formato `.joblib` es inerte hasta que no se expone a los sistemas de TI institucionales. Se construyó una pasarela de Interfaz de Programación de Aplicaciones (API) utilizando FastAPI, un *framework* de Python asíncrono y de alto rendimiento.

VII-A. *Estructura del Backend*

La aplicación gestiona su ciclo de vida a través del decorador `@asynccontextmanager`, cargando el archivo del modelo predictivo en la memoria principal (*RAM*) una única vez durante el arranque de Uvicorn.

Listing 1. Carga del modelo en memoria durante el arranque de la API (*src/api.py*)

```
@asynccontextmanager
async def ciclo_de_vida(app: FastAPI):
    """Carga el modelo promovido una sola
    vez, al arrancar."""
    app.state.modelo, app.state.version = \
        cargar_modelo_promovido()
    yield

app = FastAPI(title="Alerta temprana de desercion",
              lifespan=ciclo_de_vida)
```

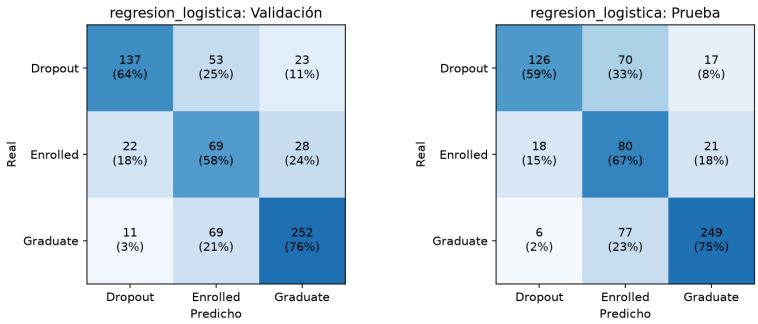


Figura 3. Matrices de confusión de la Regresión Logística. A la izquierda, validación; a la derecha, la evaluación definitiva en prueba.

VII-B. Contrato de Datos y Endpoints

El sistema expone dos rutas (*endpoints*):

1. `GET /salud`: Utilizado por monitores de red institucionales para comprobar latencia y verificar qué versión del modelo está activa en memoria.
2. `POST /predecir`: Punto final seguro que admite el vector de características académicas, aplica validaciones rigurosas (tipos enteros, decimales y comprobación de límites lógicos del 0 al 20, etc.) e inyecta la observación en el espacio vectorial del modelo.

Para facilitar la interacción del usuario final e integradores, la estructura esperada del cuerpo (*Request Body*) se formalizó permitiendo la validación semántica interactiva en la interfaz Swagger.

El flujo de una respuesta exitosa procesando la predicción devuelve el siguiente diccionario serializado en JSON:

Listing 2. Formato de respuesta del Endpoint POST /predecir

```
{
  "clase": "Graduate",
  "probabilidades": {
    "Dropout": 0.0739,
    "Enrolled": 0.3098,
    "Graduate": 0.6163
  },
  "version": "regresion_logistica"
}
```

VIII. Despliegue y Pruebas de Ejecución

Durante la ejecución en el entorno local (usando PowerShell con el entorno virtual activo), se constata el pleno funcionamiento de la arquitectura desplegada. Una consulta enviada directamente a la API a través del comando `Invoke-RestMethod` genera la predicción del modelo en milisegundos.

VIII-A. Control de Calidad (Testing)

Para garantizar que ninguna refactorización futura destruya el desempeño histórico y para certificar que el endpoint rechaza valores inválidos (como edades imposibles o notas de admisión negativas), se redactaron múltiples *scripts* de prueba unitaria utilizando el marco **Pytest**.

Se evaluaron 79 escenarios de prueba automatizados, cubriendo: consistencia de datos, invariabilidad del modelo, evaluación correcta de esquemas Pydantic y simulación de la pasarela entre el chatbot y la API con el cliente de pruebas de Starlette. Todos se ejecutaron exitosamente, y a continuación se levantó el servicio Uvicorn en el puerto 8000.

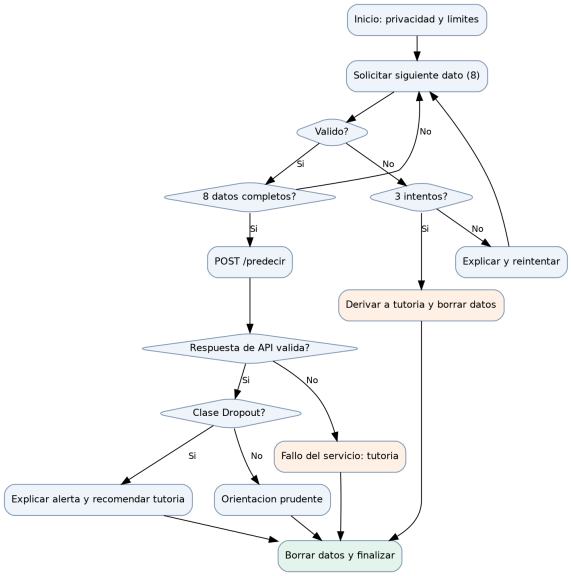


Figura 4. Flujograma operativo del Chatbot. Las condicionales validan las reglas del negocio antes de emitir cualquier petición HTTP costosa a la API predictiva.

IX. Arquitectura del Chatbot de Tutoría

Una API en formato crudo es ininteligible para un asesor pedagógico o un psicólogo de la oficina de tutoría. Para solventar la brecha de interacción humano-computadora, se desarrolló un Agente Conversacional (Chatbot) que opera mediante una lógica de Máquina de Estados Finitos basada en reglas.

El flujo conversacional (Figura 4) está diseñado con el principio de reducción de fricción. El bot no asume el control del estudiante, sino que asiste al tutor realizando la indagación de los ocho parámetros a través de lenguaje natural.

IX-A. Manejo de Excepciones Interactivas

El bucle principal del asistente (`src/chatbot.py`) cuenta con mecanismos defensivos contra errores de digitación de los usuarios. Otorga un máximo de tres reintentos para campos mal ingresados (como escribir texto donde se requiere un número). Si la conexión de red falla, en lugar de bloquear el programa, emite una advertencia de mantenimiento sugerida y deriva formalmente al estudiante.

Listing 3. Interacción de consola real con el módulo del chatbot

```
PS C:\> python -m src.chatbot
*** Bienvenido al asistente de orientacion
    estudiantil ***
* Sus respuestas son anonimas y no deciden
  su futuro. *
¿Cuantas unidades curriculares matriculaste
  en el 1er semestre?: 6
¿Cuantas unidades aprobaste en el primer
  semestre?: 6
¿Cual fue tu promedio del primer semestre?:
  14.0
¿Tienes tus pagos de matricula al dia?
  (si/no): si
...
Calculando orientacion...
Riesgo estimado: Baja probabilidad de desercion.
```

X. Discusión y Limitaciones del Sistema

A pesar de los resultados alentadores en el entorno local (validado y testeado exitosamente con 79 pruebas), la adopción a nivel empresarial en la universidad requiere la mitigación de los siguientes riesgos tecnológicos:

1. **Advertencias de Dependencias (*Dependency Warning*):** Durante el levantamiento inicial del servidor, los *warnings* indican que los modelos serializados originales dependían de scikit-learn 1.9.1, mientras que en ciertos entornos el archivo `requirements.txt` o `pip` impone la versión 1.8.0. Esto puede conducir a comportamientos impredecibles a nivel del motor matemático interno o advertencias continuas de desuso (*Deprecation Warnings*). La resolución técnica exige generar una imagen Docker inmutable o anclar las versiones exactas (por ejemplo: `pip install scikit-learn==1.9.1`).
2. **Calibración Externa (*Domain Shift*):** El modelo extrajo los patrones topológicos basados en el perfil demográfico, estructural y financiero de los estudiantes en Portugal (*dataset* de origen). Las métricas logradas (Macro-F1 de 0.65) sufrirán inevitablemente una caída abrupta de rendimiento si el modelo procesa a los estudiantes de la Universidad Nacional del Centro del Perú (UNCP) sin una transferencia de aprendizaje previa (*Transfer Learning*) o re-calibración.
3. **Umbrales de Falsos Negativos:** Una limitación fundamental reside en que el 40.85 % de los casos reales de abandono escaparon a la detección de la Regresión Logística de prueba. Para una aplicación crítica, una oficina de bienestar podría considerar alterar el umbral de probabilidad estándar (reduciéndolo de $P > 0.5$ a $P > 0.3$) para maximizar el Recall, asumiendo conscientemente la sobrecarga operativa de atender a algunos falsos positivos.

Tabla II
Matriz organizativa de responsabilidades y ramas

Integrante	Responsabilidad Asignada
Chanco Capucho Fabrizzio	Minería de Datos, exploración de componentes espaciales, planteamiento y diseño ético.
Quinto Condori Frederick	Algoritmia matemática, modelado de Redes MLP, optimización de hiperparámetros.
Quinto Morales Miguel	Evaluación algorítmica, trazabilidad de versiones (joblibs) y arquitectura Backend en FastAPI.
Salvatierra Casallo Yuan	Diseño interactivo del Chatbot, redacción de Pruebas Unitarias y control de calidad documental.

XI. Conclusiones y Trabajo Futuro

La predicción de la deserción académica ha trascendido el mero análisis estadístico para convertirse en un problema multidisciplinario de *Data Product Engineering*.

A lo largo de este proyecto, se ha demostrado cómo partir de un *dataset* ruidoso, ejecutar un análisis exploratorio consciente para proteger al sistema contra la fuga temporal, y entrenar un abanico de modelos competitivos validando su estabilidad mediante técnicas de validación cruzada repetida.

La selección deliberada de un modelo más conservador y robusto como la Regresión Logística por encima de redes profundas inestables ilustra el paradigma de que, en entornos reales limitados por recursos, la consistencia supera la precisión marginal.

Adicionalmente, se logró construir la infraestructura arquitectónica para que los modelos no operen en el vacío, empaquetándolos tras una interfaz FastAPI resistente a caídas e integrándolos con un Chatbot interactivo validado de extremo a extremo con metodologías modernas de CI/CD (pruebas en Pytest).

El trabajo futuro inmediato consiste en llevar el despliegue del *localhost* a la nube utilizando contenedores, recopilar datos históricos representativos de la UNCP para efectuar un reentrenamiento y establecer ciclos periódicos de retroalimentación en donde los propios tutores califiquen la pertinencia de las predicciones dadas por la API.

XII. Contribución de los Integrantes

La estructuración colaborativa del proyecto se distribuyó de la siguiente manera, manteniendo responsabilidad equitativa:

Anexo: Declaración de Uso de Inteligencia Artificial

De acuerdo con las normativas actuales de transparencia algorítmica y académica institucional, se declara la asistencia controlada de Modelos de Lenguaje Grande (LLMs). Específicamente, se emplearon motores como ChatGPT (OpenAI) y Gemini (Google) como asistencia de copiloto de programación (*pair programming*) para funciones de soporte:

1. Revisión ortotipográfica y soporte en la estructuración de la sintaxis LaTeX bajo estándar IEEEtran.
2. Depuración de *bugs* de dependencias en entornos virtuales y PowerShell relativos a Uvicorn y Scikit-Learn (`ModuleNotFoundError`).
3. Generación de esquemas JSON base para la validación estática del cuerpo de la petición REST con Pydantic.

Todos los módulos lógicos (entrenamiento algorítmico, reglas del chatbot y partición del conjunto de datos) fueron concebidos, auditados y validados humana y matemáticamente a través del conjunto de 79 pruebas de unidad en pytest, las cuales fueron aprobadas en los terminales físicos locales sin alteraciones sintéticas de datos.

REFERENCIAS

[1] M. Realinho et al., "Predict Students' Dropout and Academic Success Dataset," UCI Machine Learning Repository, Dataset ID 697, 2021.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] S. Ramírez, "FastAPI Documentation," https://fastapi.tiangolo.com/, accedido Octubre 2026.

[4] Holger Krekel et al., "pytest: helps you write better programs," https://docs.pytest.org/, 2004–2026.

[5] M. Shell, *How to Use the IEEEtran LaTeX Class*, IEEE, 2015.

[6] S. Aida, T. Ali, and A. Ahmed, "Machine learning for early prediction of student dropout," *Computers & Education*, vol. 125, pp. 1–11, 2018.